**Una plataforma agentica capaz de generar aplicaciones Java completas a partir de requisitos en lenguaje natural, optimizando el consumo de tokens y contexto**.

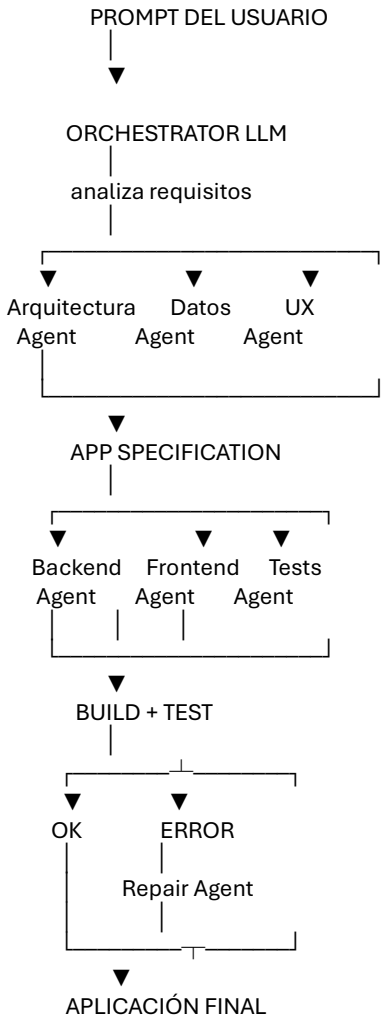
**Diseño y evaluación de un harness multiagente basado en LLM para la generación automática y eficiente de aplicaciones software Java. Optimización del consumo de contexto en sistemas multiagente basados en LLM aplicados a la generación automática de software.**

**Qué haría el sistema**

El usuario podría escribir algo como:

> *Quiero una aplicación web educativa para niños de 6 a 8 años para aprender las tablas de multiplicar mediante juegos.*
> *Debe tener usuarios, niveles, puntuaciones, recompensas y un panel para profesores.*

Y nuestro sistema transformaría eso automáticamente en:




El resultado sería un proyecto real, por ejemplo:

```
math-kids/
│
├── backend/
│   ├── Spring Boot
│   ├── REST API
```

```
│   ├── Spring Security
│   ├── JPA
│   └── PostgreSQL
│
├── frontend/
│   ├── React
│   └── TypeScript
│
├── tests/
│   ├── JUnit
│   └── frontend tests
│
├── docker-compose.yml
├── README.md
└── specification/
```

Lo verdaderamente interesante es que sea un **harness optimizador del contexto**.

Porque un sistema ingenuo haría esto:

*Agente backend:*
*recibe todo el proyecto*

*Agente frontend:*
*recibe todo el proyecto*

*Agente tests:*
*recibe todo el proyecto*

*Agente seguridad:*
*recibe todo el proyecto*

*Agente reviewer:*
*recibe todo el proyecto*

Si tienes:

Proyecto = 80.000 tokens

5 agentes × 80.000 tokens

≈ 400.000 tokens

Y en varias iteraciones nos vamos fácilmente a millones, por tanto, nuestro sistema haría justo lo contrario.

## 1. La idea importante: Context Budget

Cada agente recibe **solamente aquello que necesita**. Por ejemplo, si el agente tiene que modificar:

*UserController.java*

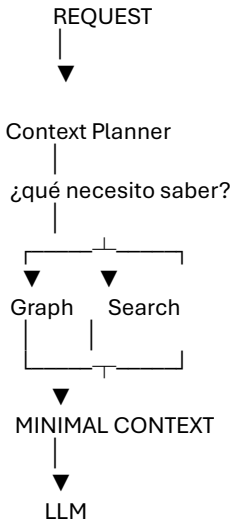
no recibe:

*todo backend*
*todo frontend*
*tests*
*README*
*SQL*
*Docker*
...

Recibe:

*TAREA*
+
*UserController*
+
*UserService interface*
+
*UserDTO*
+
*endpoints relacionados*
+
*reglas relevantes*

Algo parecido a:




## 2- Token Budget Manager

Se crearía un componente específico:

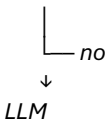
*TokenBudgetManager*

Cada tarea tendría un presupuesto:

```
{
  "task": "Crear endpoint registro de usuario",
  "maxContextTokens": 8000,
  "maxGenerationTokens": 3000,
  "maxAgentCalls": 4
}
```

Antes de invocar un LLM:

*Context Planner*
↓
*calcula relevancia*
↓
*ranking artefactos*
↓
*selecciona contexto*
↓
*¿supera presupuesto?*
|
/— *sí → compactar*




Y se guardaría métricas:

*Input tokens*
*Output tokens*
*Cached tokens*
*Número de llamadas*
*Coste estimado*
*Tiempo*
*Errores de compilación*
*Tests superados*

Eso nos permitiría demostrar experimentalmente si nuestra estrategia realmente funciona.

Podemos comparar tres arquitecturas:

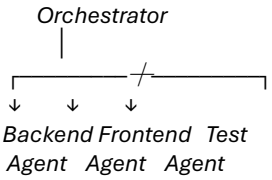
- **Estrategia A — Baseline**

  Un único agente recibe todo.

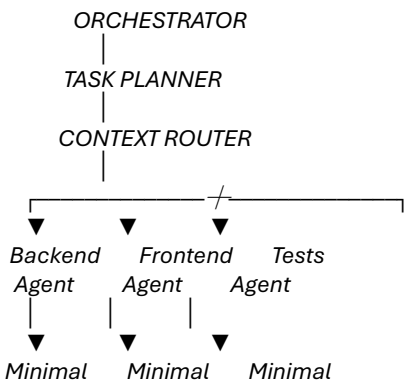


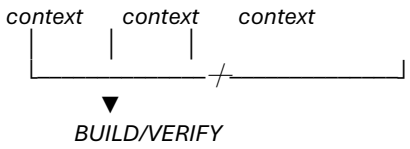

- **Estrategia B — Multiagente clásica**

  Cada agente recibe mucho contexto.




**Estrategia C — Nuestra propuesta**






Y se mediría:

| Sistema | Tokens | Coste | Build | Tests | Tiempo |
|---|---|---|---|---|---|
| Single Agent | 840k | X € | 75% | 71% | 18 min |
| Multiagent native | 1.2M | X € | 88% | 84% | 25 min |
| **EcoAgent Forge** | **310k** | **X €** | **91%** | **89%** | **14 min** |

Por tanto, como aportación medible, calcularíamos la reducción en % de nuestro sistema al generar aplicaciones manteniendo o mejorando la calidad.

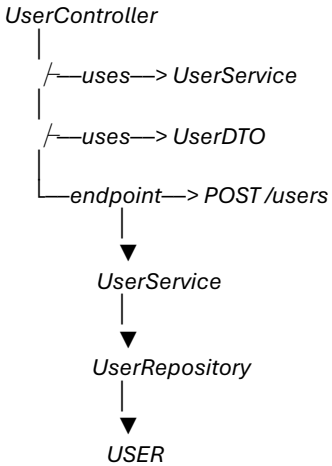
## 3- Context Slice

Se puede añadir a algo parecido al graph.json, en lugar de meter todo el conocimiento directamente en el prompt, se puede convertir en una estructura consultable.

Nuestro nuevo harness podría generar automáticamente:

Project Knowledge Graph

Por ejemplo:




Cuando el agente necesite modificar UserController, se recorrería el grafo depth = 2 y se recuperaría solo:

UserController
UserService
UserDTO
UserRepository

No los otros 150 archivos.

Incluso lo podríamos llamar Context Slice. Formalmente podríamos definir:

```
ContextSlice(T) =
  RelevantCode(T)
 + RelevantRequirements(T)
 + Dependencies(T)
 + PreviousDecisions(T)
```

donde:

```
T = tarea actual
```

Y después intentar minimizar:

```
min Tokens(ContextSlice(T))
```

sujeto a:

```
CompilationSuccess >= threshold

TestSuccess >= threshold

RequirementCoverage >= threshold
```

## 4- Reducir conversación entre agentes

Este es otro problema típico.

No haría:

```
Backend Agent:
"He implementado esto porque..."

Frontend Agent:
"Gracias. Yo creo que..."

Backend:
"Estoy de acuerdo..."
```

Eso quema tokens sin aportar demasiado. Usarímos **artefactos estructurados**.

Por ejemplo:

```
{
 "task": "USER-004",
 "status": "DONE",
 "created": [
  "UserController.java",
  "UserService.java"
 ],
 "modified": [
  "SecurityConfig.java"
 ],
 "api": [
  "POST /api/users"
 ],
 "decisions": [
  "email must be unique"
 ]
}
```

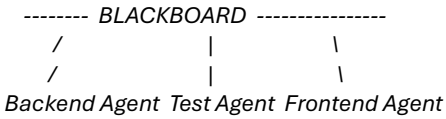
El siguiente agente consume eso, no una conversación de 3.000 tokens.

## 5- Shared Blackboard

En vez de agentes hablándose directamente:

*Agent A ↔ Agent B ↔ Agent C*

tendríamos:




El Blackboard podría guardar:

*requirements.json*

*architecture.json*

*tasks.json*

*decisions.json*

*api-contract.yaml*

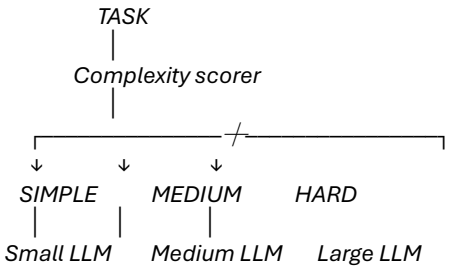
*database-schema.json*

*project-graph.json*

Los agentes consultan únicamente lo necesario, esto reduce contexto y además hace el sistema mucho más reproducible.

## 6- Model Router

No todas las tareas necesitan el modelo más potente.




Por ejemplo:

*Crear getter*
*→ modelo pequeño*

*Generar DTO*
*→ modelo pequeño*

*Diseñar arquitectura*
*→ modelo potente*

*Resolver error complejo*
*→ modelo potente*

Eso te permite optimizar no solo tokens sino también:

*coste*
*latencia*
*energía*

## 7- Evitar regenerar archivos completos

Es un punto crítico, Supongamos que tenemos:

*UserService.java*

con 600 líneas, el agente cambia 4.Una implementación sencilla pediría al LLM devolver las 600 pero nuestro harness trabajaría con:

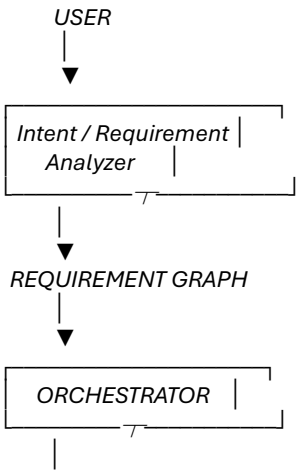
```
@@ UserService.java

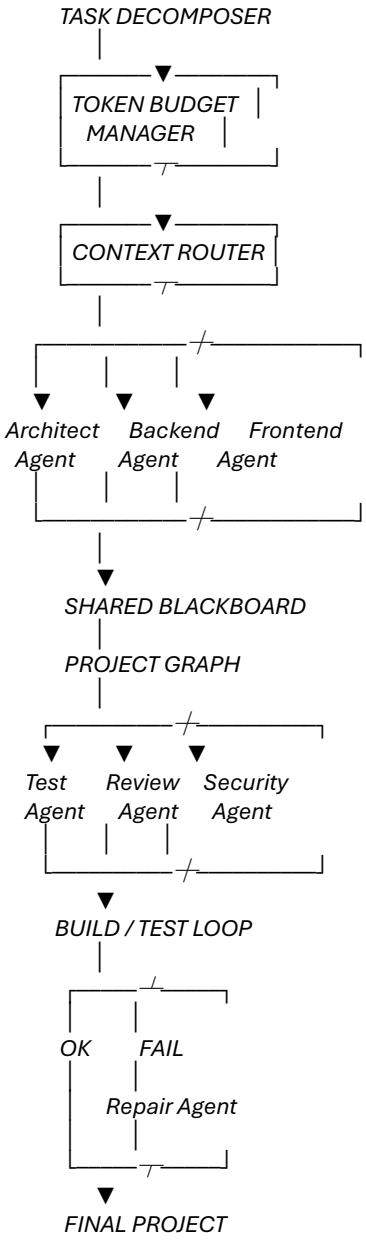
+ public User findByEmail(String email) {
+    return repository.findByEmail(email)
+       .orElseThrow(UserNotFoundException::new);
+ }
```

Esto reduce mucho output tokens.

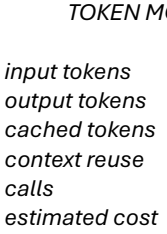
## CONCLUSIÓN

**La arquitectura que propondríamos finalmente**






Y transversalmente:




La utilización de selección dinámica de contexto, memoria estructurada, especialización de agentes y generación incremental permite reducir significativamente el consumo de tokens durante la generación autónoma de aplicaciones, sin degradar su calidad funcional.